Graph Unfolding Approach for HLS Dependency Analysis

Fachbereich Informatik
Zhang,Mingshen

TECHNISCHE
UNIVERSITÄT
DARMSTADT

1. Introduction

This report presents a structured methodology for graph unfolding, a technique that transforms cyclic dependency graphs in High-Level Synthesis (HLS) into a time-expanded acyclic representation across k iterations. The primary objective is to enable advanced scheduling optimizations—such as loop pipelining and parallelism extraction—by explicitly modeling temporal dependencies in a static analysis framework.
The unfolding process consists of two key phases, each addressing distinct challenges in dependency preservation and tool compatibility:
Unfolding Phase:
Node Replication: Each node in the original graph is replicated k times, corresponding to its execution across successive iterations. This creates an expanded time-indexed graph where nodes are uniquely identified by their iteration instance (i = 0 to k−1).
Edge Redirection: Inter-iteration dependencies are captured by redirecting edges to reflect temporal lags. For example, a cyclic edge ($A \rightarrow B$) in the original graph becomes a set of acyclic edges ($A_i \rightarrow B_{i+1}$) ensuring correct scheduling semantics.
Post-Processing Phase:
Iteration Annotation: Nodes are labeled with metadata (e.g., iteration IDs, timing offsets) to facilitate downstream tool integration.
Format Standardization: The unfolded graph is transformed into a tool-agnostic intermediate representation (e.g., XML or DOT format), ensuring compatibility with HLS schedulers and optimizers.
By decoupling cyclic dependencies into an acyclic, time-aware model, this approach provides a formal foundation for analyzing and optimizing iterative constructs in HLS, while maintaining correctness guarantees.Approach and Design Decisions

2. Graph Unfolding Algorithm

The unfold_graph algorithm operates on a directed graph (DOT format input) to systematically transform cyclic dependencies into a time-expanded acyclic structure across k iterations. Below is a detailed breakdown of its key mechanisms:

2.1. Node Replication

Procedure:
Each node in the original graph is replicated k times, generating temporally indexed instances (e.g., n3 → n3_0, n3_1, ..., n3_{k-1}).
Design Rationale:
Traceability: The suffix-based naming convention (_0, _1, etc.) preserves the original node identifier, enabling backward tracing of dependencies during debugging or optimization.
Temporal Semantics: Each suffix explicitly encodes the node’s temporal position within the unfolded execution timeline, critical for modeling multi-cycle operations in HLS.

2.2. Edge Handling

Edges are categorized and processed based on their constraint semantics to ensure correct temporal dependencies:
Non-Constraint Edges (constraint=false):
Behavior: These edges represent cross-iteration dependencies (e.g., loop-carried dependencies).
Transformation:
Edges originating from self-dependent nodes are redirected to future or past iterations to break cycles.
For an edge A -> B in the original graph, the unfolded graph generates edges A_i -> B_{(i+ Δ) mod k}, where Δ is derived from the dependency’s latency (e.g., Δ =1 for sequential iterations).
Purpose: Ensures cyclic behaviors are resolved into schedulable, acyclic dataflows.
Constraints Edges:
Behavior: Represent intra-cycle dependencies (e.g., combinational logic within an iteration).
Transformation: Edges remain anchored to the same cycle (delta = 0), preserving temporal locality (e.g., A_i -> B_i).

2.3. Attribute Management

Label Sanitization:
Quotes in labels are temporarily removed and reinserted to comply with DOT syntax rules, preventing parsing errors in downstream tools.
Attribute Pruning:
Non-critical visual attributes (e.g., color, style) are stripped from constraint edges in intermediate cycles (i > 0) to reduce graph clutter while retaining functional metadata (e.g., latency, constraint).
Cycle-Specific Retention:
Attributes of the original edge are preserved only in the final cycle (e.g., n3_2 -> n5_0) to maintain backward compatibility with legacy analysis tools.2.2 Post-Processing (`process_second_pass`)

The second pass reformats node labels into a structured string `iteration:label_value;` and inserts them into the DOT file. This enables tools to map nodes to their original operations and cycles.

- Design Decision:

  The regex-based extraction (`(\S+)\s+\[label="([^"]+)"\]`) balances robustness against variations in DOT formatting.

3. Problems Encountered

3.1.1. Initial Confusion:

During the initial implementation of the graph unfolding technique, I encountered ambiguity in interpreting the core rules governing dependency redirection. Key unresolved questions included:

Edge Scope Determination: How to distinguish nodes requiring cross-iteration edges (e.g., n_i → n_{i+1}) from those limited to intra-iteration dependencies.

Index Shift Representation: The lecture materials lacked explicit visualization of temporal index shifts in unfolded graphs, making it difficult to map theoretical concepts to practical implementation.

Correctness Criteria: No clear verification framework was provided to validate whether the unfolded graph preserved semantic equivalence to the original cyclic dependencies.

3.1.2. Node Naming Strategy:

Mistake: Initially, I don't reserve the origin nodename like moodelexample

Resolution: Retaining original node names (e.g., n7 [label="459:ISUB"];) proved essential for traceability, enabling direct comparison between the original and unfolded graphs to verify structural consistency.

Cyclic Dependency Identification:

3.1.3. Key Realization

Nodes with self-referential edges (e.g., n3 → n3) serve as anchors for determining cross-iteration redirection logic. These cyclic dependencies explicitly dictate where edges must span multiple iterations ($\Delta > 0$).

4. Experimental Results

4.1. Test Case1: ADPCMn-decode-771-791

- Input: A dependency graph (`ADPCMn-decode-771-791.dot`) with 4 nodes and 6 t edges.
- Unfolding Parameters: `k = 3`.

Results:

1. Node Count: Increased from 4 to 12 (3 cycles × 4 nodes).
2. Edge Count:
   - Original: 6 edges (2 constraint, 4 non-constraint).
   - Unfolded: 18 edges (6 constraint, 12 non-constraint).
3. Validation:
   Correctly unfolding.

4.2. Test Case2:ADPCMn-decode-425-472

- Input: A dependency graph (`ADPCMn-decode-425-472.dot`) with 11 nodes and 15 edges.
- Unfolding Parameters: `k = 3`.

Results:

1. Node Count: Increased from 11to 33 (3 cycles × 11 nodes).
2. Edge Count:
   - Original: 15 edges (7 constraint, 8 non-constraint).
   - Unfolded: 45 edges (21 constraint, 24 non-constraint).
3. Validation:
   Correctly unfolding.

5. Conclusion

The unfolding algorithm successfully transforms cyclic dependency graphs into time-expanded representations while preserving semantic constraints. Key challenges included attribute management and cycle boundary conditions, which were addressed through structured naming and conditional logic. Future work includes integrating this component into a full HLS scheduling pipeline.